

Claude Sonnet 4.5: An Engineering Guide to Advanced Prompting and Agentic Design

Executive Summary

The release of Anthropic's Claude Sonnet 4.5 on September 29, 2025, marks a pivotal moment in the evolution of large language models, particularly for complex, long-horizon tasks in domains like software engineering, finance, and cybersecurity.¹ Positioned as the world's leading model for agentic tasks, coding, and computer use, Sonnet 4.5 is not merely an incremental upgrade; it represents a fundamental shift in the human-model interaction paradigm.³ This model demands a more rigorous, architectural approach to prompt engineering, moving beyond simple natural language instructions to the design of structured, explicit, and state-aware control systems.

This report provides a comprehensive, developer-focused guide to mastering Claude Sonnet 4.5. It is based exclusively on technical documentation, system cards, and expert analyses published on or after September 15, 2025. The analysis is structured into four key areas: foundational interaction principles, advanced reasoning techniques, system instruction architecture, and agentic design patterns. It synthesizes official best practices with community-driven findings to offer a holistic framework for harnessing the model's full capabilities.

Key findings indicate that Sonnet 4.5's power is unlocked through a combination of explicit instruction, structural prompt design, and the strategic use of its advanced features, such as Extended Thinking and the new context management APIs. The model's engineered directness and precision, while highly effective, place a greater burden on the developer to provide complete and unambiguous specifications. Success with Sonnet 4.5 is less about crafting clever prose and more about architecting robust, reusable prompt scaffolds that can reliably guide the model through complex, multi-step workflows. This guide provides the principles, patterns, and specific techniques required to build these sophisticated applications and unlock the next generation of AI-powered automation.

Section 1: Foundational Principles of Interaction with Claude 4.5

Interacting with Claude Sonnet 4.5 requires a deliberate departure from the conversational and often inferential style effective with previous model generations. The model's architecture prioritizes precision, directness, and structural clarity. Mastering these foundational principles is a prerequisite for leveraging the more advanced agentic and reasoning capabilities discussed later in this report.

1.1 The Paradigm of Explicitness: Moving Beyond Inference

Claude 4 models, and Sonnet 4.5 in particular, have been trained for precise and literal instruction following.⁴ This represents a significant design choice away from models that might attempt to "go above and beyond" or infer user intent from ambiguous prompts. Consequently, the onus of clarity and completeness has shifted decisively to the user.

A primary best practice is to frame requests as direct commands rather than suggestive questions. For tasks involving tool use or file modification, this distinction is critical. A prompt phrased as, "Can you suggest some changes to improve this function?" will likely yield only a list of recommendations. In contrast, the command, "Implement these changes to improve this function's performance," will compel the model to take direct action.⁴

Furthermore, performance is substantially enhanced when the model is provided with the rationale behind an instruction. This allows it to generalize the underlying principle rather than merely adhering to a surface-level rule. For example, the instruction "NEVER use ellipses" is far less effective than the more contextual command: "Your response will be read aloud by a text-to-speech engine, so never use ellipses since the text-to-speech engine will not know how to pronounce them".⁴ This level of explicitness helps the model understand constraints and goals, leading to more targeted and appropriate responses.

1.2 Structural Integrity: The Power of XML and Formatting

Claude models are trained to recognize and prioritize instructions and context provided within XML tags.⁶ This feature is not a mere formatting preference but a core mechanism for structuring prompts to reduce ambiguity and ensure reliable interpretation. Using tags such as <context>, <instructions>, <example>, and <output_format> allows developers to create a clear, machine-parsable hierarchy within the prompt, delineating background information from actionable commands and specifying the desired structure of the response.⁴

Beyond explicit tagging, Sonnet 4.5 exhibits a strong tendency to mirror the formatting style of the prompt itself. A prompt written in simple prose with minimal markdown will elicit a response in a similar style. Conversely, a highly structured prompt using headings and lists may encourage a similar format in the output.⁴ This mirroring behavior can be used as a subtle, implicit method for controlling the output's presentation, complementing the more explicit control offered by XML tags.

1.3 Engineered Personality: The Terse and Direct Nature of Sonnet 4.5

A consistent observation among both official documentation and community reports is that Sonnet 4.5 is "noticeably more terse" and direct than its predecessors.⁵ It often proceeds directly to implementation or task execution without the conversational pleasantries or affirmations common in other models.

This behavior is not an emergent quirk but a deliberate design feature. Analysis of leaked system prompts reveals explicit, hard-coded rules instructing the model to avoid "unnecessary affirmations or filler phrases like 'Certainly!', 'Of course!', 'Absolutely!'".¹⁰ This directive appears to be a strategic choice to counter the perceived "sycophancy" of other AI assistants and to optimize for information density in interactions. This engineered terseness has profound implications for prompt design; to work effectively with the model, user prompts should be similarly direct, concise, and focused on the task at hand.

1.4 The Implications of Engineered Behavior

The confluence of precise instruction-following, a requirement for explicit context, and an engineered, terse personality creates a model that is exceptionally powerful but can also appear "brittle" or "sloppy" if not prompted with sufficient rigor. The model does *exactly* what it is told, with minimal interpretive leeway. This means any ambiguity, omission, or logical flaw in the prompt is more likely to be reflected in the output.

This dynamic becomes clear when examining user reports. While Anthropic's documentation highlights the model's superior instruction-following³, some developers on community forums describe its performance on complex refactoring tasks as "junior-level output" or "sloppy," noting that it can miss crucial steps like updating all file references or writing necessary tests.⁹ These are not necessarily failures of the model's capability but rather direct consequences of its core design. The model is executing the provided instructions with maximum efficiency and minimal assumption. If the prompt does not explicitly state "update all references across the entire codebase and verify with existing test harnesses," the model may perform a more localized change that fulfills the literal, albeit incomplete, request.

This behavior signifies a critical shift in the nature of prompt engineering for advanced models. The prompt is no longer a mere suggestion or query; it is a formal specification document. The burden of ensuring completeness, defining edge cases, and specifying acceptance criteria has moved almost entirely to the developer. This elevates the importance of standardized, reusable "prompt scaffolds" from a helpful best practice to a fundamental necessity for building reliable, production-grade applications on top of Claude Sonnet 4.5.¹²

Section 2: Mastering Advanced Reasoning and Thinking Techniques

Claude Sonnet 4.5 provides developers with unprecedented control over its cognitive processes through a suite of API-level features. These "thinking techniques" transform the model from a black-box text generator into a more transparent and steerable problem-solving partner. Choosing the appropriate technique—whether it's allocating a dedicated "thinking budget," enabling reasoning between actions, or guiding it manually—is crucial for tackling complex challenges.

2.1 Extended Thinking: Allocating a "Thinking Budget"

At its core, Sonnet 4.5 is a hybrid reasoning model, offering both a default mode for fast responses and an "extended thinking mode" for computationally intensive problems.¹³ This mode allows the model to dedicate a specific allocation of tokens—a "thinking budget"—to internal reasoning, planning, and exploring different approaches before generating a final answer.

This feature is enabled via the Messages API by including a thinking object in the request body, with type set to "enabled" and budget_tokens specifying the desired token allocation.¹⁵ While API control is the primary method, some user interfaces may offer simpler triggers; one user reported receiving an email from Anthropic suggesting that including the word "think" in a prompt could activate the mode.¹⁷

Extended thinking is most valuable for tasks where depth and accuracy are paramount over low latency. Ideal use cases include complex mathematical proofs, physics problems, competitive coding challenges, and detailed document analysis.¹⁸ For example, financial analysts have found it delivers "investment-grade insights that require less human review" when applied to complex risk analysis.² When activated, the model produces visible "thinking" blocks in the API response that articulate its step-by-step reasoning. For Claude 4 models, this visible output is often a summary of a much larger, more detailed internal thought process, providing the full intelligence benefit without overwhelming the user with raw data.¹³

2.2 Interleaved Thinking: Reasoning Between Actions

Interleaved Thinking is a powerful, beta-level extension of the extended thinking paradigm. Where extended thinking focuses on deep, upfront reasoning, interleaved thinking allows the model to inject reasoning steps *between* tool calls and actions within a single turn.¹⁵ This capability is a game-changer for complex agentic workflows, as it enables the model to dynamically adjust its strategy based on the real-time results of its actions.

Activation requires adding the beta header interleaved-thinking-2025-05-14 to the API request, in addition to enabling extended thinking.¹⁵ With this mode active, an agent can execute a workflow with a level of adaptiveness that was previously impossible. For instance, a cybersecurity agent could:

1. Search system logs for errors (first tool call).
2. Receive the log data.
3. **Engage in a thinking step** to analyze the logs and hypothesize about a potential root cause.
4. Check the status of a related service to verify its hypothesis (second tool call).
5. Receive the service status.
6. **Engage in another thinking step** to confirm the cause and formulate a patch.
7. Apply the patch (third tool call).

Each of these thinking steps is visible in the API response, providing a complete audit trail of the agent's decision-making process.¹⁵

2.3 Chain-of-Thought (CoT) Prompting: A Manual Alternative

In environments where API-level control is unavailable or for simpler tasks, manual Chain-of-Thought (CoT) prompting remains a highly effective technique for improving reasoning and accuracy.⁷ This method involves explicitly instructing the model to "think" through the problem step-by-step as part of its response. There are three primary levels of sophistication:

1. **Basic CoT:** Simply append the phrase "Think step-by-step" to the end of a prompt. This encourages the model to break down its reasoning but offers no specific guidance on how to do so.¹⁶
2. **Guided CoT:** Provide a high-level outline of the steps the model should follow in its reasoning process. For example, "First, analyze the user's past behavior. Second, identify key product features. Third, draft a personalized message connecting the two".¹⁶
3. **Structured CoT:** This is the most robust method for programmatic use. It employs XML tags to clearly separate the reasoning process from the final answer. The prompt instructs the model to place its reasoning within <thinking> tags and the final deliverable within a separate tag, such as <answer> or <email>. This allows an application to easily parse the response, display the final answer to an end-user, and log the thought process for debugging or analysis.¹⁶

2.4 Comparison of Thinking Techniques

Choosing the correct reasoning technique depends on the complexity of the task, the environment (API vs. chat), and the need for dynamic adaptiveness. The following table provides a comparative overview to guide this decision.

Technique	Activation Method	Cost Model	Key Capability	Ideal Use Case
Extended Thinking	API parameters (thinking object)	Billed as dedicated "thinking tokens"	Deep, upfront reasoning and problem decomposition for a single,	Solving difficult math or science problems; in-depth

			complex request.	document analysis; generating a complex code function in one shot.
Interleaved Thinking	API beta header + Extended Thinking	Billed as dedicated "thinking tokens"	Dynamic, adaptive reasoning that occurs <i>between</i> multiple tool calls within a single turn.	Multi-step agentic workflows; interactive debugging; complex research involving multiple data sources.
Chain-of-Thought (CoT)	Instruction within the user prompt	Billed as standard output tokens	Manually guided, explicit reasoning that is part of the main response body.	Improving accuracy in environments without API control; simple multi-step logic; ensuring response transparency.

Section 3: Architecting System Instructions and Behavioral Guardrails

System instructions, or system prompts, represent the highest level of control over a model's behavior. They are persistent instructions that remain active throughout a conversation, establishing the model's persona, operational boundaries, and default decision-making logic. For Claude Sonnet 4.5, designing effective system instructions is less about creative writing and more about architectural design, creating a stable control plane for the model's operation.

3.1 Core Principles from Leaked System Prompts

Analysis of leaked system prompts for Claude 4 family models provides invaluable insight into the core design principles that govern their behavior.⁸ These are not merely suggestions but foundational rules that can be emulated in custom system prompts:

- **Directness and Anti-Sycophancy:** As noted previously, the model is explicitly instructed to avoid filler words and conversational affirmations. It is also trained to be skeptical and avoid flattery, responding directly to the substance of a query.⁸
- **Style Mirroring:** The system prompt instructs the model to match its response style to the user's query: "Claude should give concise responses to very simple questions, but provide thorough responses to complex and open-ended questions".⁸
- **Prescriptive Formatting:** The prompts contain highly specific rules about output formatting. For instance, they often mandate writing in prose and paragraphs, avoiding bullet points or numbered lists unless explicitly requested by the user. This is a clear design choice to differentiate its output style from competitors known for overusing lists.¹⁰
- **Contextual Self-Awareness:** The system prompt typically includes variables for the model's identity and the current date and time (e.g., The assistant is Claude... The current date is {{currentDateTime}}). This provides essential temporal and self-referential context for all interactions.¹⁹

3.2 A Framework for Reusable System Scaffolds

For any production application, especially those used by a team, relying on ad-hoc prompts is inefficient and unreliable. The best practice is to develop standardized, reusable system prompt "scaffolds" that ensure consistent behavior.¹² A robust scaffold should contain several core components, creating a comprehensive operational contract for the model¹²:

1. **Persona/Role:** Defines the expertise and perspective the model should adopt. (e.g., Role: Senior engineer.)
2. **Objective:** A clear, high-level statement of the agent's primary goal. (e.g., Objective: Implement feature X with minimal risk.)
3. **Constraints:** Hard boundaries and rules the model must not violate. (e.g., Constraints: Only modify files listed by me. Propose a step-by-step plan; wait for approval before code changes.)
4. **Acceptance Criteria:** A definition of "done" for a given task. (e.g., Acceptance: New

tests in tests/<area>/ must cover A/B. All lints pass.)

5. **Output Format:** A strict definition of the desired output structure, often using XML tags. (e.g., Output format: PLAN: numbered steps. DIFFS: unified diffs per file.)

3.3 Implementing Dynamic Behavioral Guardrails

System prompts can go beyond defining a static persona to programming the model's dynamic decision-making logic. This is achieved by providing explicit instructions, often within XML tags, that act as behavioral guardrails.

A key example is controlling the model's default level of autonomy. For ambiguous user requests, the model can be configured to be either proactive or conservative ⁴:

- **Proactive Guardrail:** Using a tag like `<default_to_action>`, one can instruct the model to infer the most likely useful action and proceed with it, rather than asking for clarification. This is useful for building more autonomous agents.
- **Conservative Guardrail:** Conversely, a tag like `<do_not_act_before_instructions>` can instruct the model to default to research and recommendation, requiring explicit user confirmation before making any changes. This is critical for safety in high-stakes environments.

Another crucial guardrail, particularly for agentic coding, is one that mitigates hallucinations by forcing verification. An instruction such as `<investigate_before_answering>` Never speculate about code you have not opened. If the user references a specific file, you **MUST** read the file before answering. `</investigate_before_answering>` prevents the model from making incorrect assumptions about the state of a codebase. ⁴

3.4 System Prompts as a Configurable Control Plane

The evolution of system prompts for Claude Sonnet 4.5 demonstrates a significant leap in the level of abstraction available to developers. Where traditional system prompts primarily defined a model's persona or knowledge domain, these new techniques allow for the configuration of its core operational logic. Instructions like `<default_to_action>` are not about *what* the model is, but *how it decides*.

This progression moves the system prompt from being a simple "primer" to a configurable "control plane." The prompt is no longer just describing a desired state (e.g., "you are a helpful

assistant") but is actively defining a process (e.g., "if the user's intent is ambiguous, *then* default to taking direct action"). This represents a more sophisticated form of programming the model's behavior, focusing on its decision-making heuristics rather than just its personality or knowledge base.

As agentic systems grow in complexity and autonomy, this trend is likely to continue. The "system prompt" will increasingly resemble a "policy document," where developers define intricate decision trees, rules of engagement, escalation paths for uncertainty, and operational safety policies. This makes the art of system instruction design a critical discipline not only for application performance but also for the safety, governance, and reliability of AI systems in production.

Section 4: A Handbook for Agentic Design and Long-Horizon Tasks

Claude Sonnet 4.5 is explicitly designed to power autonomous agents capable of performing complex, long-running tasks that can span hours or even days.¹ This remarkable endurance is not an innate property of the model itself but an emergent capability that arises from a combination of advanced prompting patterns, sophisticated state management, and the strategic use of the Anthropic API ecosystem. This section provides a practical handbook for architecting such agents.

4.1 The State Management Triad: Architecting Agent Memory

An agent's ability to maintain coherence and pursue a goal over a long period is entirely dependent on its ability to externalize and manage its state. A robust agent built on Sonnet 4.5 typically employs a three-tiered memory architecture.

1. **Layer 1: The Filesystem as a Scratchpad:** For short-term, intra-session memory, Sonnet 4.5 naturally uses the local filesystem as a scratchpad. Without explicit instruction, it has been observed creating files like `progress.txt`, `SUMMARY.md`, and `tests.json` to document its work, track its plan, and store structured data.⁵ Effective prompts should not only allow this behavior but actively encourage and structure it, for example, by specifying a standard format for its progress logs.
2. **Layer 2: The Memory Tool for Persistent Knowledge:** To retain knowledge across multiple sessions or context window resets, the **Memory Tool** is essential. This API

feature, currently in public beta, allows Claude to perform CRUD (Create, Read, Update, Delete) operations on files within a dedicated, developer-managed storage directory.²¹ This enables an agent to build a persistent knowledge base over time. A critical prompting pattern for memory-enabled agents is to include a protocol in the system prompt that forces the agent to consult its memory at the beginning of every task, such as: IMPORTANT: ALWAYS VIEW YOUR MEMORY DIRECTORY BEFORE DOING ANYTHING ELSE.²¹

3. **Layer 3: Git for Checkpointing and Versioning:** In the context of software development, Git serves as the ultimate state management system. Sonnet 4.5 is particularly adept at leveraging git commands as tools. It can use git log to understand a project's history, git diff to review its own changes before committing, and git status to re-orient itself after a context reset.⁴ This makes Git an indispensable tool for checkpointing and versioning an agent's work.

4.2 Context Engineering: Managing the Conversation Window

Even with external memory, the model's immediate working context is limited by the context window. "Context engineering" is the discipline of optimally managing this finite resource during long-running tasks.

The primary tool for this is the **Context Editing API**. This feature, also in public beta and enabled via the context-management-2025-06-27 header, can be configured to automatically remove the oldest, least relevant tool call results from the conversation history as it approaches the token limit. This "pruning" preserves the main conversational flow while freeing up space for new information, effectively extending the agent's operational lifespan.¹²

This API feature is complemented by Sonnet 4.5's native **context awareness**. The model is able to track its available token budget and can be prompted to act accordingly.⁴ A powerful prompting pattern is to combine this awareness with the memory tool, instructing the agent on how to gracefully handle an impending context limit: "As you approach your token budget limit, save your current progress and state to memory before the context window refreshes. Never artificially stop any task early regardless of the context remaining".⁴

4.3 Advanced Tool Use and Orchestration

Effective agents are proficient tool users. Sonnet 4.5 exhibits an aggressive approach to tool

use, favoring parallel execution to maximize efficiency. It will often run multiple web searches or bash commands simultaneously if they are not dependent on one another.² While this boosts performance, it can cause issues in workflows that require sequential execution. Prompts must therefore explicitly manage this behavior:

- **To Encourage Parallelism:** A system prompt can include an instruction like `<use_parallel_tool_calls> If you intend to call multiple tools and there are no dependencies... make all of the independent tool calls in parallel. </use_parallel_tool_calls>`.⁴
- **To Enforce Sequential Execution:** For dependent tasks, a simple instruction like "Execute operations sequentially with brief pauses between each step to ensure stability" is effective.⁴

Finally, for any agent performing critical or irreversible actions, a **Human-in-the-Loop (HITL)** pattern is a non-negotiable safety measure. The agent's prompt scaffold must instruct it to formulate a plan of action, present it to the user, and explicitly wait for an approval signal before proceeding with execution.¹²

4.4 Agentic State Management Techniques

Choosing the right combination of memory and state management mechanisms is a critical architectural decision when designing an agent. The following table summarizes the trade-offs between the three primary techniques.

Mechanism	Persistence	Primary Use Case	Implementation	Key Prompting Strategy
Filesystem (.txt, .json)	Session-based (ephemeral)	Short-term task tracking, planning, and storing intermediate results within a single, continuous run.	Natural model behavior. Guide with prompts to enforce structure and consistency.	"Create a progress.md file to log your steps. Before each new action, read the file to review your progress."
Memory Tool	Cross-session	Building a long-term	API integration.	"Your first step is always to

API	(persistent)	knowledge base, maintaining project state across interruptions or context resets.	Requires developer-provided backend storage (e.g., S3 bucket, local directory).	use the view command of your memory tool to check for earlier progress."
Git Version Control	Permanent & Versioned	Checkpointing code changes, maintaining a verifiable history of work, and recovering from errors.	Tool integration. Requires a git executable in the agent's environment.	"After implementing the feature, run git diff to review your changes, then create a commit with a descriptive message."

Section 5: Synthesis and Strategic Recommendations

Successfully deploying Claude Sonnet 4.5 in production environments requires a strategic synthesis of the principles and techniques outlined in this report. It involves understanding the model's unique characteristics, choosing the right interaction pattern for the task, and building the necessary scaffolding to ensure reliability and safety.

5.1 The "Autonomous but Brittle" Paradox: Scaffolding for Success

The central theme emerging from the analysis of Sonnet 4.5 is a paradox: the model is capable of unprecedented autonomy, yet this autonomy is highly dependent on the quality and completeness of the instructions it receives. Its combination of speed, directness, and literal instruction-following means that it can efficiently execute a well-defined plan but may produce incomplete or "sloppy" results when given a vague or underspecified prompt.⁹

The solution to this paradox is not to create more complex narrative prompts, but to provide

the model with robust structural and procedural **scaffolding**. The most effective pattern identified is the **"test-first" approach** to development tasks. By instructing the agent to first generate a comprehensive suite of tests based on the requirements, and only then to write the implementation code to pass those tests, the developer creates a self-guiding framework. The tests serve as an unambiguous, executable specification that guides the agent's work and provides a clear definition of success, mitigating the risk of incomplete implementation.⁴

5.2 Choosing Your Interaction Model: Pair Programmer vs. Autonomous Agent

Community comparisons between Sonnet 4.5 and its contemporaries, such as GPT-5 Codex, reveal that different models excel at different interaction patterns. The choice of which model—or which prompting style—to use should be dictated by the nature of the task.¹¹

- **Claude as a Pair Programmer:** For interactive, iterative work where a human is closely guiding the process, Sonnet 4.5 is a superior choice. Its high speed provides a rapid feedback loop, its steerability over long conversations allows for nuanced control, and its preference for simple, stepwise commands makes it easier to debug and redirect than models that favor complex one-liners.¹¹ In this mode, the human provides the high-level strategy, and the model provides accelerated execution.
- **Claude as an Autonomous Agent:** For long-running, "fire-and-forget" tasks, Sonnet 4.5 is highly capable but requires more upfront investment in prompt architecture. The prompt must be a comprehensive specification document, detailing every step, constraint, and verification procedure. It is best suited for tasks that are rich in tool use and OS-level interaction, where its speed and computer-use capabilities provide a distinct advantage.²⁶

5.3 Final Recommendations for Production Deployment

1. **Standardize with Scaffolds:** Do not allow prompts to be an ad-hoc process. Develop, version, and standardize a library of prompt scaffolds for your team's most common tasks. This is the single most important step toward achieving reliable and repeatable results.¹²
2. **Measure and Monitor:** Implement robust logging and monitoring for your agentic applications. Track key metrics such as token consumption (per task and per agent), task completion rates, tool call error rates, and instances of refusal. Be particularly vigilant about token usage in multi-agent workflows, as coordination overhead can lead to

unexpectedly high costs.²⁷

3. **Leverage the Full Ecosystem:** The true power of Sonnet 4.5 is unlocked when prompt engineering is combined with the broader Anthropic ecosystem. Effective production systems will integrate the Claude Agent SDK for building blocks, the context management APIs for endurance, and developer tools like the VS Code extension for a tighter human-AI feedback loop.¹
4. **Always Keep a Human in the Loop:** For any task that involves modifying production systems, making financial transactions, or handling sensitive data, full autonomy is not advisable. The most robust and safe pattern is to have the AI agent operate up to a final approval gate. The agent's role is to analyze the situation, generate a detailed plan, and prepare the necessary changes (e.g., a pull request). The final git merge or deploy command must be executed by a human expert.¹²

Appendix: Machine-Readable Prompting Toolkit (JSON Format)

JSON

```
{
  },
  {
    "id": "api_config_interleaved_thinking",
    "name": "API Request Config: Interleaved Thinking",
    "type": "api_request_body",
    "description": "A sample API request body for enabling interleaved thinking with a specified thinking budget. Use for complex, multi-step agentic tasks requiring dynamic strategy adjustment.",
    "content": {
      "model": "claude-sonnet-4-5",
      "max_tokens": 4096,
      "thinking": {
        "type": "enabled",
        "budget_tokens": 16000
      },
    },
    "anthropic_beta_headers": [
      "interleaved-thinking-2025-05-14"
    ],
  },
}
```

```

    "messages":,
    "tools":
  },
  "source_snippets": [15]
},
{
  "id": "guardrail_prompt_investigate_first",
  "name": "Guardrail Prompt: Investigate Before Answering",
  "type": "system_prompt_fragment",
  "description": "An XML-tagged instruction to be inserted into a system prompt to prevent the model
from hallucinating about the contents of a codebase.",
  "content": "<investigate_before_answering>Never speculate about code you have not opened or
state of files you have not read. If the user references a specific file, you MUST read the file before
answering any questions about it. Ensure all your claims about the codebase are grounded in files you
have directly inspected in the current session. Give hallucination-free
answers.</investigate_before_answering>",
  "source_snippets": [4]
},
{
  "id": "agentic_prompt_context_aware_memory",
  "name": "Agentic Prompt: Context-Aware Memory Management",
  "type": "system_prompt_fragment",
  "description": "An instruction for long-running agents that leverages context awareness to gracefully
manage state before the context window is reset or compacted.",
  "content": "Your context window will be automatically managed to allow you to continue working
indefinitely. Do not stop tasks early due to token budget concerns. As you approach your token budget
limit, you will be notified. Your priority at that point is to save your current progress and state to your
external memory files before the context window refreshes. Always be as persistent and autonomous
as possible and complete tasks fully.",
  "source_snippets": [4]
},
{
  "id": "api_config_context_editing",
  "name": "API Request Config: Context Editing",
  "type": "api_request_body",
  "description": "A sample API request body for enabling automatic context editing, which clears stale
tool results to prolong agentic sessions.",
  "content": {
    "model": "claude-sonnet-4-5",
    "max_tokens": 4096,
    "anthropic_beta_headers": [
      "context-management-2025-06-27"
    ],
  },

```



```

    "context_management": {
      "edits": [
        {
          "type": "clear_tool_uses_20250919"
        }
      ]
    },
    "messages":,
    "tools":
  },
  "source_snippets": [21, 24]
}
]

```

Works cited

1. Announcing Claude Sonnet 4.5 on Vertex AI | Google Cloud Blog, accessed October 9, 2025, <https://cloud.google.com/blog/products/ai-machine-learning/announcing-claude-sonnet-4-5-on-vertex-ai>
2. Introducing Claude Sonnet 4.5 - Anthropic, accessed October 9, 2025, <https://www.anthropic.com/news/claude-sonnet-4-5>
3. Claude Sonnet 4.5 - Anthropic, accessed October 9, 2025, <https://www.anthropic.com/claude/sonnet>
4. Claude 4 prompt engineering best practices, accessed October 9, 2025, <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/claude-4-best-practices>
5. Anthropic's Claude Sonnet 4.5 is Live in Cline - Cline Blog, accessed October 9, 2025, <https://cline.bot/blog/claude-sonnet-4-5>
6. Best Practices for Prompting Claude : r/ClaudeAI - Reddit, accessed October 9, 2025, https://www.reddit.com/r/ClaudeAI/comments/1biukoy/best_practices_for_prompting_claude/
7. Prompt engineering overview - Claude Docs, accessed October 9, 2025, <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>
8. A Guide to the Claude 4 and ChatGPT 5 System Prompts - Forte Labs, accessed October 9, 2025, <https://fortelabs.com/blog/a-guide-to-the-claude-4-and-chatgpt-5-system-prompts/>
9. Claude 4.5 experiences : r/Anthropic - Reddit, accessed October 9, 2025, https://www.reddit.com/r/Anthropic/comments/1nu5sm2/claude_45_experiences/
10. Here's the Exact System Prompt That Kills Filler Words in Sonnet 4.5 : r/ClaudeAI - Reddit, accessed October 9, 2025, https://www.reddit.com/r/ClaudeAI/comments/1ntssvo/heres_the_exact_system_prompt_that_kills_filler_words_in_sonnet_45/

- [prompt_that_kills_filler/](#)
11. Tested GPT-5 Codex vs Claude Sonnet 4.5 vs Kimi K2 on a real ..., accessed October 9, 2025,
https://www.reddit.com/r/ClaudeCode/comments/1nwtv4j/tested_gpt5_codex_vs_claude_sonnet_45_vs_kimi_k2/
 12. Claude 4.5 Integration Best Practices for Developers (2025), accessed October 9, 2025,
<https://skywork.ai/blog/claude-4-5-integration-best-practices-developers-2025/>
 13. Claude Sonnet 4.5 System Card - Anthropic, accessed October 9, 2025,
<https://www.anthropic.com/claude-sonnet-4-5-system-card>
 14. Getting Started with Claude 4.5: Beginner Guide for Everyday Users - Skywork.ai, accessed October 9, 2025, <https://skywork.ai/blog/claude-4-5-beginner-guide/>
 15. Claude Sonnet 4.5 Extended Thinking Explained: Can Code for 30 ..., accessed October 9, 2025,
<https://medium.com/@cognidownunder/claude-sonnet-4-5-4ddf33d53cd4>
 16. Building with extended thinking - Claude Docs, accessed October 9, 2025,
<https://docs.claude.com/en/docs/build-with-claude/extended-thinking>
 17. Claude 4.5 Sonnet: lots of hype, middling ranks. What gives? : r ..., accessed October 9, 2025,
https://www.reddit.com/r/ClaudeAI/comments/1nwup8t/claude_45_sonnet_lots_of_hype_middling_ranks_what/
 18. Using extended thinking | Claude Help Center, accessed October 9, 2025,
<https://support.claude.com/en/articles/10574485-using-extended-thinking>
 19. Claude 4 System Prompts : Operational Blueprint and Strategic Implications - Medium, accessed October 9, 2025,
<https://medium.com/@tuhinsharma121/decoding-claude-4-system-prompts-operational-blueprint-and-strategic-implications-727294cf79c3>
 20. AI Prompting : r/ClaudeAI - Reddit, accessed October 9, 2025,
https://www.reddit.com/r/ClaudeAI/comments/1nk0kin/ai_prompting/
 21. Managing context on the Claude Developer Platform \ Anthropic, accessed October 9, 2025, <https://www.anthropic.com/news/context-management>
 22. Managing context on the Claude Developer Platform - Anthropic, accessed October 9, 2025, <https://anthropic.com/news/context-management>
 23. Everything You Need to Know about Claude 4.5 - PromptHub, accessed October 9, 2025,
<https://www.prompthub.us/blog/everything-you-need-to-know-about-claude-4-5>
 24. Context editing - Claude Docs, accessed October 9, 2025,
<https://docs.claude.com/en/docs/build-with-claude/context-editing>
 25. Claude 4.5 Sonnet is here : r/singularity - Reddit, accessed October 9, 2025,
https://www.reddit.com/r/singularity/comments/1ntnegj/claude_45_sonnet_is_here/
 26. Claude Sonnet 4.5 vs GPT-5-Codex: what real developers say | by ..., accessed October 9, 2025,
<https://open-data-analytics.medium.com/claude-sonnet-4-5-vs-gpt-5-codex-wh>

[at-real-developers-say-24fe0787d0c8](#)

27. Claude 4.5 Explained: Key Technical Improvements for AI Users, accessed October 9, 2025,

<https://skywork.ai/blog/claude-4-5-explained-key-ai-technical-features/>